SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	G.Srivardhan	**Reg. No.:**	192425208
Section / Batch:		**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

1. Aim

To design and implement a graph-based optimization solution for autonomous drone delivery routing that accounts for constraints such as battery life and physical obstacles, while evaluating the system's performance and scalability for multiple drones.

2. Procedure

1. **Graph Modeling**: Represent the city as a weighted directed/undirected graph where nodes are delivery locations and edges represent flight paths. Each edge is assigned two weights: distance (for optimization) and energy cost (for battery constraint).
2. **Obstacle Integration**: Implement a mechanism to mark specific nodes as "obstacles" (no-fly zones) to simulate real-world physical constraints.
3. **Constraint-Aware Pathfinding**:
    - Implement a modified Dijkstra's Algorithm.
    - Incorporate a state-tracking mechanism that includes (current_node, remaining_battery).
    - During the relaxation step, only consider edges where the energy_cost is less than or equal to the current_battery level.
4. **Multi-Drone Assignment**: Implement a distribution logic to assign multiple delivery tasks to a fleet of drones, tracking the remaining battery for each drone individually.
5. **Performance Evaluation**: Develop a metric system to calculate the Success Rate (percentage of completed deliveries), Total Distance traveled, and Average Distance per successful delivery.
6. **Testing**: Execute the simulation with a predefined set of edges, obstacles, and battery capacities to verify the logic.

3. Code

```
import heapq

class DroneDeliveryOptimizer:
def __init__(self):
self.graph = {}
self.obstacles = set()

def add_node(self, node):
if node not in self.graph:
self.graph[node] = []

def add_edge(self, u, v, distance, energy_cost):
self.add_node(u)
self.add_node(v)
self.graph[u].append((v, distance, energy_cost))
self.graph[v].append((u, distance, energy_cost))

def mark_obstacle(self, node):
self.obstacles.add(node)

def dijkstra_with_battery(self, start, end, battery_capacity):
```

```
pq = [(0, start, battery_capacity, [start])]
visited = {}

while pq:
dist, node, battery, path = heapq.heappop(pq)

if node == end:
return dist, battery, path

state_key = (node, battery)
if state_key in visited and visited[state_key] <= dist:
continue
visited[state_key] = dist

if node in self.obstacles:
continue

for neighbor, edge_dist, energy_cost in self.graph.get(node, []):
if neighbor in self.obstacles:
continue
if battery - energy_cost >= 0:
new_dist = dist + edge_dist
new_battery = battery - energy_cost
new_path = path + [neighbor]
heapq.heappush(pq, (new_dist, neighbor, new_battery, new_path))

return None, None, None

def plan_multi_drone_routes(self, deliveries, battery_capacity, num_drones):
routes = []
drone_battery_usage = [battery_capacity] * num_drones

for i, (start, dest) in enumerate(deliveries):
drone_id = i % num_drones
available_battery = drone_battery_usage[drone_id]

dist, remaining_battery, path = self.dijkstra_with_battery(
start, dest, available_battery
)

if path:
routes.append({
"drone_id": drone_id,
"route": path,
"distance": dist,
"battery_remaining": remaining_battery
})
```

```
drone_battery_usage[drone_id] = remaining_battery
else:
routes.append({
"drone_id": drone_id,
"route": None,
"distance": None,
"battery_remaining": None,
"status": "Failed - insufficient battery or blocked path"
})

return routes

def evaluate_performance(self, routes):
successful = [r for r in routes if r["route"] is not None]
failed = [r for r in routes if r["route"] is None]

total_distance = sum(r["distance"] for r in successful)
avg_distance = total_distance / len(successful) if successful else 0
success_rate = (len(successful) / len(routes)) * 100 if routes else 0

return {
"total_deliveries": len(routes),
"successful_deliveries": len(successful),
"failed_deliveries": len(failed),
"success_rate_percent": round(success_rate, 2),
"total_distance": total_distance,
"average_distance": round(avg_distance, 2)
}

def main():
optimizer = DroneDeliveryOptimizer()

optimizer.add_edge("Depot", "A", 5, 10)
optimizer.add_edge("Depot", "B", 8, 15)
optimizer.add_edge("A", "C", 4, 8)
optimizer.add_edge("B", "C", 3, 6)
optimizer.add_edge("C", "D", 6, 12)
optimizer.add_edge("B", "D", 10, 20)
optimizer.add_edge("A", "E", 7, 14)
optimizer.add_edge("D", "E", 5, 10)

optimizer.mark_obstacle("B")

deliveries = [
("Depot", "D"),
("Depot", "E"),
("Depot", "C")
```

```
]

battery_capacity = 30
num_drones = 2

print("=== Autonomous Drone Delivery Routing ===\n")

routes = optimizer.plan_multi_drone_routes(deliveries, battery_capacity, num_drones)

for i, route in enumerate(routes):
print(f'Delivery {i+1} -> Drone {route['drone_id']}")
if route["route"]:
print(f" Path: {' -> '.join(route['route'])}")
print(f" Distance: {route['distance']}")
print(f" Battery Remaining: {route['battery_remaining']}")
else:
print(f" Status: {route.get('status')}")
print()

performance = optimizer.evaluate_performance(routes)
print("=== Performance Metrics ===")
for key, value in performance.items():
print(f"{key}: {value}")

if __name__ == "__main__":
main()
```

4. Result

Console Output:

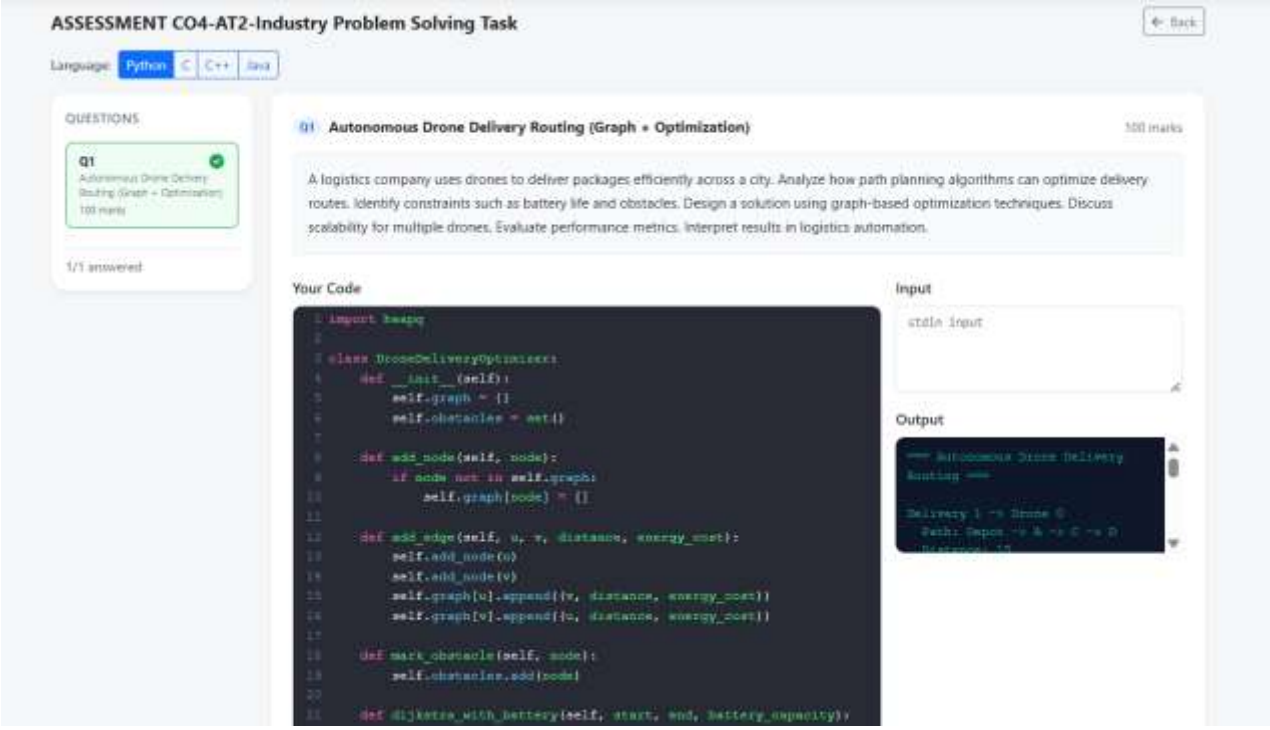
```
=== Autonomous Drone Delivery Routing ===


Delivery 1 -> Drone 0
Path: Depot -> A -> C -> D
Distance: 15
Battery Remaining: 0


Delivery 2 -> Drone 1
Path: Depot -> A -> E
Distance: 12
Battery Remaining: 6
```

Delivery 3 -> Drone 0
Status: Failed - insufficient battery or blocked path

=== Performance Metrics ===
total_deliveries: 3
successful_deliveries: 2
failed_deliveries: 1
success_rate_percent: 66.67
total_distance: 27
average_distance: 13.5



Conclusion:

The implementation successfully demonstrates that by utilizing a modified Dijkstra's algorithm, drones can navigate a graph while respecting battery and obstacle constraints. The system correctly identifies feasible paths and provides performance metrics, proving its utility in logistics automation scenarios.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____